Computing as an Engineering Tool

Engineering Computation with Python — Chapter 1

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why Engineers Compute
2. Programming Fundamentals
3. Flowcharts
4. Pseudocode

© Muním Zabidi

Table of Contents

1 Why Engineers Compute

2 Programming Fundamentals

3 Flowcharts

4 Pseudocode

Why Does an Engineer Need to Compute at All?

Consider one of the first equations you meet in electrical engineering, Ohm's law:

$$V = IR, \qquad \text{or equivalently} \qquad I = \frac{V}{R}.$$

A 12 V supply is connected across a 470 Ω resistor. You need the current. There are three ways to get it.

By hand	With a calculator	With Python
$I = \frac{12}{470} \approx 25.5\,\text{mA}$ Fast for one resistor. Leaves no record.	Key in 12 ÷ 470. Faster still. Nobody — including you, next week — can check what you did.	`I = V / R` Looks like more work … for one resistor.

What if there are 10 000 measurements?

A data logger records voltage once per second for nearly three hours. You need the current at every instant, the average power, the maximum current, and a plot.

- By hand: impossible.
- With a calculator: impossible.
- With Python: a few lines — and the same few lines work for ten measurements or ten million.

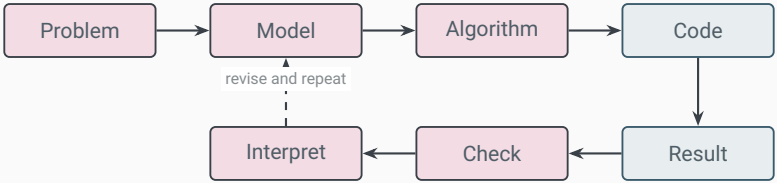
The point

Programming is not a separate skill added on top of engineering; it is what makes engineering calculations scale from the textbook to the laboratory.

© Muním Zabidi

The Engineering Computation Cycle

Every worked example in this course, from this slide to the final chapter, follows the same cycle:



The two steps engineers most often skip

Check and **Interpret**. A computer executes wrong instructions just as precisely as right ones — a result that has not been checked is not yet an answer.

© Muním Zabidi

An LED needs a series resistor to limit its current. Supply 5 V, LED drop 2 V, desired current 20 mA:

$$R = \frac{V_{\text{supply}} - V_{\text{LED}}}{I}.$$

```
V_supply = 5.0   # volts
V_led    = 2.0   # volts
I_led    = 0.02  # amps

R = (V_supply - V_led) / I_led
print("R =", R, "ohms")
```

```
R = 150.0 ohms
```

Check. Volts ÷ amps = ohms. Magnitude is plausible for an LED series resistor.

Interpret. 150 Ω may not be a standard value — fit the nearest available (150 Ω or 220 Ω) and accept a slightly different current.

© Muním Zabidi

The Bench Experiment Used Throughout This Course

One real measurement, returned to in every chapter, so each new technique applies to data you already understand.

The experiment. A 100 μF capacitor, charged to 12 V, discharges through a 4.7 kΩ resistor. Logged every 20 ms for two seconds: time, voltage, current, temperature.

$$v(t) = V_0\, e^{-t/\tau}, \qquad \tau = RC = 0.47\,\text{s}.$$

Why one dataset. A genuine recording, not a tidy formula — it has noise, three dropped samples near $t = 0.8$ s, and one corrupted temperature reading.

These flaws are not accidents: they are why several later chapters exist (data quality, curve fitting, calculus).

Table of Contents

1 Why Engineers Compute

2 **Programming Fundamentals**

3 Flowcharts

4 Pseudocode

We define a computer, simply, as one that computes: a programmable electronic device that can store, retrieve, and process data.

Many everyday actions already follow a logical sequence of steps — turning a page of a book, by hand:

1. Lift hand.
2. Move hand to the right side of the book.
3. Grasp the top-right corner of the page.
4. Move the hand from right to left until positioned.
5. Let go of the page.

Definition

Describing the order and sequence of operations in a process is called programming. The list of steps a computer follows is a program.

© Muním Zabidi

The Three Phases of Program Development

Phase 1: Problem Solving

- Analysis and specification
- General solution (algorithm)
- Verify

Phase 2: Implementation

- Concrete solution (program)
- Test

Phase 3: Maintenance

- Use
- Maintain

A program is rarely finished

These three phases form a cycle. A program is usually not complete just because it runs correctly once: requirements change, and new errors are discovered during real use.

A computer program and an algorithm look similar because all programs are algorithms — a program is simply an algorithm written in a form a computer can execute.

Informal definition

An algorithm is a verbal or written description of a logical sequence of actions — a recipe, a set of instructions, a set of directions.

Four equivalent ways to describe an algorithm:

1. Step-by-step diagrams (e.g. origami instructions)
2. Flowcharts — boxes and arrows
3. A list of plain-language steps
4. Pseudocode — resembles a real language, tied to none

© Munim Zabidi

Formal definition

An algorithm is an ordered set of unambiguous, executable steps that defines a terminating process.

Every valid algorithm shares these characteristics:

- Each instruction is precise and unambiguous.
- A finite number of steps.
- Produces a correct result.
- Does not depend on any particular programming language.

A note on history. The word “algorithm” comes from the name of Al-Khwarizmi, a Persian mathematician who first proposed the concept in the 9th century.

© Muním Zabidi

Example: Starting a Car

1. Insert the key.
2. Depress the brake pedal.
3. Make sure the transmission is in Park (or neutral).
4. Turn the key to the start position.
5. If the engine starts within six seconds, release the key to the ignition position.
6. If the engine doesn't start, release the key and the gas pedal, wait ten seconds, and repeat steps 3–6, but not more than five times.
7. If the car still doesn't start, call the garage.

Notice

Every step is unambiguous, and the sequence includes a decision (step 5) and a repeat (step 6) — the two control structures beyond plain sequence.

The Four Control Structures

Regardless of language, every algorithm is built from just four control structures:

Sequence

Statements executed one after another. Nothing branches, nothing repeats.

Loop (repetition/iteration)

`WHILE condition DO s1` — repeats while the condition holds.

Selection (branch/decision)

`IF condition THEN s1 ELSE s2` — one of two paths, chosen by a condition.

Subprogram

A named, reusable group of the three structures above (procedure / function / method).

Flowcharts and pseudocode both exist to express these four structures.

© Muním Zabidi

Table of Contents

1 Why Engineers Compute

2 Programming Fundamentals

3 **Flowcharts**

4 Pseudocode

What Is a Flowchart?

A flowchart is a graphical representation of the problem-solving steps of a program, drawn so the logic is clear.

System flowchart	Program flowchart
Gives the complete processing mechanism of a system. Cannot be converted directly into a program. *Example:* the organizational structure of a college.	Gives the problem-solving method for a specific task. Can be converted into a program. *Example:* a procedure to calculate the area of a triangle.

This course uses *program* flowcharts throughout.

Flowchart Symbols

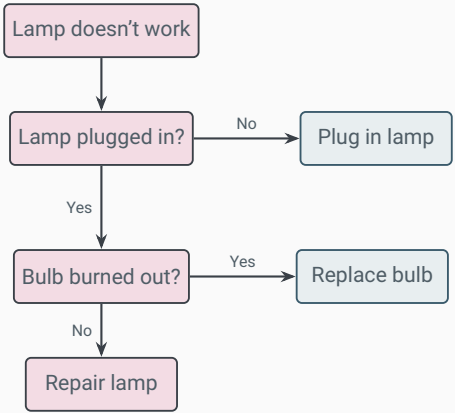
- **Rectangles** — processing steps.
- **Diamonds** — points of branch/decision.
- **Parallelograms** — input and output steps.
- **Circles** — connectors.
- **Ovals** — start and finish of the flowchart.
- **Arrows** — direction of flow between symbols.

A good flowchart

One start point and one end point; flow lines that do not intersect; no programming-language-specific syntax; neat and clear for the reader.

© Munim Zabidi

Worked Example: Diagnosing a Lamp



A diamond decision at each stage, rectangle actions, flow merging back to a single path — the pattern every flowchart in this course follows.

© Munim Zabidi

Table of Contents

1 Why Engineers Compute

2 Programming Fundamentals

3 Flowcharts

4 **Pseudocode**

Pseudocode: A Preview

Integer division of *n* by *d*, using only subtraction:

```
procedure IntegerDivision
input : n, d
output: result
begin
    result <- 0
    while n >= d
        n <- n - d
        result <- result + 1
    end
    return result
end
```

Notice

A `while` loop (repetition) containing a sequence of two statements — exactly the control structures from the previous slide, written in a language-independent notation.

Tracing Pseudocode by Hand: IntegerDivision

Trace the `IntegerDivision` procedure (previewed earlier) with $n = 6, d = 2$. Initial values: $n = 6, d = 2, result = 0$.

- **Iteration 1:** $6 \geq 2 \rightarrow$ True. $n = 4, result = 1$.
- **Iteration 2:** $4 \geq 2 \rightarrow$ True. $n = 2, result = 2$.
- **Iteration 3:** $2 \geq 2 \rightarrow$ True. $n = 0, result = 3$.
- **Exit:** $0 \geq 2 \rightarrow$ False. Loop ends.

Result

Return $result = 3$ — and indeed $6 \div 2 = 3$.

Tracing a procedure by hand — writing down how every variable changes on every pass — is the same skill used later to debug real Python code.

This chapter answered one question: why and how engineers compute. The rest of this course builds on it directly.

- Chapters 2–5: Python itself — variables, data types, control flow, functions.
- Chapter 6 onward: arrays, plotting, and the bench dataset reappearing as the running example.
- Every worked example still follows the same cycle: Problem → Model → Algorithm → Code → Result → Check → Interpret.

Next: Your First Python Programs

© Muním Zabidi